

Proyecto Final: The DOPO Hardest Game

Universidad Escuela Colombiana de Ingeniería Julio Garavito

Ingeniería de sistemas

Desarrollo Orientado Por Objetos

Estudiantes:

**Juan Pablo Cuervo Contreras,
David Felipe Ortiz Salcedo**

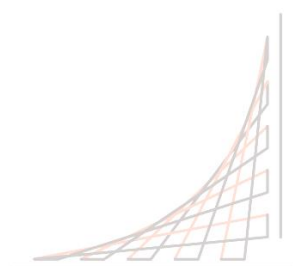
Docentes:

**María Irma Díaz Roza,
Santiago Rocha Durán**

16 de mayo de 2026

Tabla de Contenido

RETROSPECTIVAS	2
Retrospectiva – Entrega 1	3
Retrospectiva – Entrega 2	4
CICLOS DEFINIDOS	7



ESCUELA
COLOMBIANA
DE INGENIERÍA
JULIO GARAVITO

VIGILADA MINEDUCACIÓN

UNIVERSIDAD

RETROSPECTIVAS

Retrospectiva – Entrega 1

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

R/: **Objetos del juego:** se definió los distintos objetos como lo fueron el enemigo, jugador, pared y moneda.

Zonas seguras: se definieron las zonas seguras con el que el jugador puede comenzar la partida y acabar la misma.

Crear niveles (configuraciones): por ahora sólo se re-creó el primer nivel del juego original para esta entrega y se añadió el que el jugador pueda cargar niveles que haya guardado.

Lógica y visualización del juego: para realizar The DOPO Hardest Game, se tuvo en cuenta varias herramientas visuales como lo fue el uso de una paleta hecha por Claude para ver mejor el cómo poner las paredes, enemigos, zonas seguras, jugador y monedas en una pantalla, y comandos para la eficiencia en el código. De manera que los métodos tuvieran pocas responsabilidades y tener en cuenta lo que se aprendió durante el semestre como lo es el uso de interfaces, clases abstractas, etc.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

R/:

El estado actual del proyecto es terminado en los mini-ciclos definidos con el tiempo suficiente para poder realizar los menús, definir los enemigos y jugador (movimiento, estados, obtener moneda, rebotar en paredes, entre otros).

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

R/:

- **Juan Pablo Cuervo Contreras:** 34 horas
- **David Felipe Ortiz Salcedo:** 34 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/:

Consideramos que el mayor logro fue haber podido hacer que los enemigos tuvieran colisiones con el jugador y la pared de manera que se utilizaron comandos/métodos que ofrece Java para hacer esta acción.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/:

El mayor problema técnico fue que a veces los niveles no cargaban, pero esto dependía de donde estaban guardados los niveles en cada equipo de los integrantes, sin embargo, simplemente era cambiar la ruta donde estaba el archivo .txt.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R/:

Pensar en cómo realizar el trabajo, es decir, dar distintas ideas de modo que coincidiéramos con el fin de plasmarlas en el proyecto. Nos comprometemos a mejorar nuestra comunicación y claridad, debido a que hubo algunas confusiones durante el desarrollo de esta primera entrega.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

R/:

Las prácticas más útiles fueron Planning: The project is divided into iterations, Coding: All production code is pair programmed, Coding: Code must be written to agreed standards y Coding: Only one pair integrates code at a time. La práctica más útil fue Coding: Only one pair integrates code at a time dado que, al ser un proyecto con una entrega en corto tiempo, la mejor decisión fue que a veces, cada uno revisara por su lado lo que le faltaba al proyecto.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R/:

- W3Schools. W3Schools — Tutoriales y referencias de programación.
<https://www.w3schools.com/>
- Rodríguez, A. (s. f.). Documentar proyectos Java con Javadoc: Comentarios, símbolos, tags (deprecated, param, etc.) Aprender a Programar.
https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=646:documentar-proyectos-java-con-javadoc-comentarios-simbolos-tags-deprecated-param-etc-cu00680b&catid=68&Itemid=188

Retrospectiva – Entrega 2

COLOMBIANA

EDUC

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

R/: Los mini-ciclos se organizaron según la evolución real del proyecto:

- Motor base y modelo jugable, Estructura del dominio, Creación de jugadores y enemigos, Construcción de niveles desde archivos.
- Comportamientos de enemigos y modos base.
- Primer flujo jugable: Se justifican porque antes de trabajar en interfaz era necesario tener una base funcional: jugador, enemigos, monedas, paredes, colisiones, niveles y reglas mínimas de victoria, Interfaz gráfica, menú y selección de partida.
- Ventana principal y panel de juego: Menú basado en estados. Selección de modalidad. Personalización del jugador. Selección de nivel.
- Observadores de interfaz: Se justifican porque el proyecto necesitaba separar la lógica del juego de la presentación visual. Aquí se aplicó el aislamiento visual: la UI escucha al juego mediante Observer y se organiza con State, Progreso, HUD, pausa, victoria y persistencia
- HUD del juego: Tiempo configurable y derrota. Pantalla de nivel completado. Redimensionamiento del render. Pausa y navegación. Guardado y carga con Memento.
- Player vs Player y selección del segundo jugador: Se justifican porque después de tener motor e interfaz, hacía falta mejorar la experiencia de juego: mostrar información, permitir progresión, pausar, cargar partidas y soportar dos jugadores, Estabilización, pruebas y correcciones
- Runner de pruebas: Pruebas de recolección y modos. Pruebas de tiempo, victoria y reinicio. Pruebas de colisiones y zonas seguras. Pruebas de Factory, Builder y HUD.

Correcciones de estabilidad. Verificación final. Se justifican porque al final era necesario comprobar que las reglas principales funcionaran y corregir errores detectados durante la integración.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

R/:

El proyecto está en: estabilización, pruebas y correcciones.

La mayoría de mini-ciclos principales ya están completados: el motor funciona, los niveles se construyen desde archivos, existe interfaz gráfica, hay menú, personalización, HUD, pausa, victoria, derrota por tiempo, modo Player vs Player, carga inicial con Memento y pruebas unitarias.

Se considera que está en estabilización porque ya no se está construyendo la base del juego, sino verificando, corrigiendo y puliendo funcionalidades. De hecho, ya existe un runner de pruebas con 10 casos que validan recolección de monedas, colisiones, tiempo, victoria, Builder, Factory, movimiento de enemigos y render básico del HUD.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

R/:

- **Juan Pablo Cuervo Contreras:** 44 horas
- **David Felipe Ortiz Salcedo:** 44 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/:

El mayor logro fue integrar los patrones de diseño en una arquitectura funcional y mantenible, no solo mencionarlos de forma teórica.

El proyecto quedó dividido de forma clara: la interfaz está en presentation/ui, la creación de entidades está en factory, la construcción de niveles en level/builder, los comportamientos en strategy, los estados del menú en state y el guardado en save/memento.

Esto es importante porque permite extender el juego sin romper lo anterior. Por ejemplo, agregar un nuevo tipo de enemigo implica crear una nueva estrategia de movimiento y conectarla a la fábrica, no modificar todo el motor.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/:

El mayor problema técnico fue integrar la interfaz con el motor sin mezclar responsabilidades.

Al inicio era fácil que el menú, el render y la lógica del juego quedaran acoplados. Para resolverlo, se separó la UI en estados de menú, se usó MenuData para transportar las decisiones del usuario y se conectó la interfaz con el motor mediante métodos claros en MainWindow y TheDopoHardestGame.

También se usó Observer para que elementos visuales como el HUD, el temporizador y las pantallas de victoria reaccionaran al estado del juego sin controlar directamente las reglas internas.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R/:

Pensar en cómo realizar el trabajo, es decir, dar distintas ideas de modo que coincidiéramos con el fin de plasmarlas en el proyecto. Nos comprometemos a mejorar nuestra comunicación y claridad, debido a que hubo algunas confusiones durante el desarrollo de esta primera entrega.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

R/:

Las prácticas más útiles fueron Planning: The project is divided into iterations, Coding: All production code is pair programmed, Coding: Code must be written to agreed standards y Coding: Only one pair integrates code at a time. La práctica más útil fue Coding: Only one pair integrates code at a time dado que, al ser un proyecto con una entrega en corto tiempo, la mejor decisión fue que a veces, cada uno revisara por su lado lo que le faltaba al proyecto.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R/:

- W3Schools. W3Schools — Tutoriales y referencias de programación.
<https://www.w3schools.com/>
- Rodríguez, A. (s. f.). Documentar proyectos Java con Javadoc: Comentarios, símbolos, tags (deprecated, param, etc.) Aprender a Programar.
https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=646:documentar-proyectos-java-con-javadoc-comentarios-simbolos-tags-deprecated-param-etc-cu00680b&catid=68&Itemid=188

Retrospectiva – Entrega 3

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

R/: Los mini-ciclos definidos fueron los siguientes:

- **Re-estructuración del proyecto:** Se re-nombraron las carpetas para cumplir con los requisitos: domain, presentation y test para una mejor organización del proyecto.
- **Arreglo de errores:** Se corrigieron errores como el incumplimiento de polimorfismo o sobrecarga de métodos. Así mismo, se arregló el hecho de que la capa de dominio utilizara la capa de presentación.
- **Cambio general en la UI del usuario:** Se mejoró la visualización del jugador para poder elegir su skin y su nivel preferidos, además de añadir un game-over por si el tiempo termina en cualquier modo de juego y si un jugador gana, se muestra en la ventana cuando se va a cambiar de nivel. Por consiguiente, se hizo una pequeña modificación a ciertos niveles para hacerlos más justos con respecto al modo PvP.
- **Refactorización del código de acuerdo con las indicaciones del profesor:** Se consideraron las retrospectivas dadas por el profesor Santiago Rocha con el fin de simplificar la utilización y menor complicación del código (a pesar de que aún falta por pensar si se deben utilizar ciertos patrones de diseño).
- **Implementación del modo de juego PvP:** Se diseñó e implementó el nuevo modo de juego que se pide como requisito para esta entrega: un jugador empieza al inicio y otro al final, ambos deben obtener las monedas en el menor tiempo posible, si ambos chocan entonces se suma 1 muerte y deben comenzar desde su propio inicio.

- **Implementación del guardado y abrir partidas:** Se añadió el guardar y abrir partidas por si el jugador quiere jugar sus propias partidas o si desea guardar el estado del juego.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

R/:

El proyecto actualmente está en su fase intermedia, a pesar de haber implementado las nuevas características que este debe ofrecer, el manejo del código e implementación del diseño aún debe mejorar para un mejor entendimiento y desarrollo sencillo a la hora de extender o querer saber algún concepto que no conozcamos.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

R/:

- **Juan Pablo Cuervo Contreras:** 36 horas
- **David Felipe Ortiz Salcedo:** 36 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/:

Consideramos que el mayor logro fue haber podido diseñar la intersección entre los dos jugadores y que las monedas desaparezcan una vez que ambos hayan obtenido las monedas, dado que pensamos que, si sólo dejábamos que un jugador obtuviera las monedas, no tendría sentido principalmente el modo pvp y la competencia en que el jugador obtenga las monedas en el menor tiempo posible.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/:

El mayor problema técnico que encontramos fue diseñar un PlayerSnapshot lo suficientemente completo como para capturar todo el estado relevante del jugador, sin romper el encapsulamiento ni acoplar demasiado las clases entre sí. Al principio era fácil olvidar alguna variable (por ejemplo, un modificador de velocidad o un flag temporal) y eso generaba inconsistencias al restaurar la partida.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R/:

En esta entrega mejoramos demasiado en la comunicación a pesar de ciertas indiferencias y entendimientos confusos, al haberlo hablado con el profesor Santiago Rocha, se mejoró la comunicación en el equipo con el fin de que cada uno supiera qué hizo, cuándo lo hizo y por qué lo hizo de una manera que tal vez que al uno o al otro no les haya gustado; esto, con el fin de que ambos podamos escucharnos y tener mejor entendimiento como se dijo anteriormente. Nos comprometemos a repasar mucho más el código que estamos realizando en el proyecto para poder sentirnos seguros y poder exponérselo a cualquiera persona que lo pueda preguntar, así mismo, a pesar de usar IA preferimos optar por entender primero y escuchar la opinión mutua.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

R/:

- W3Schools. W3Schools — Tutoriales y referencias de programación.
<https://www.w3schools.com/>
- Rodríguez, A. (s. f.). Documentar proyectos Java con Javadoc: Comentarios, símbolos, tags (deprecated, param, etc.) Aprender a Programar.
https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=646:documentar-proyectos-java-con-javadoc-comentarios-simbolos-tags-deprecated-param-etc-cu00680b&catid=68&Itemid=188
- Refactoring Guru. (s.f.). *Patrones de diseño*. <https://refactoring.guru/es/design-patterns>

CICLOS DEFINIDOS

Ciclo 1:

- Completar la Integración de Elementos Especiales
- Añadir listas List y List en Level.java.
- Actualizar ConcreteLevelBuilder.java para instanciar e inyectar BOMBA y FUENTE desde los archivos .txt.
- Añadir método checkSpecialElements() en GameEngine.update() para manejar las muertes por bombas y vidas extras por fuentes.
- Actualizar GamePanel.java para dibujar los elementos especiales. 2. Desarrollar la Inteligencia Artificial (Modo PvsM)

Ciclo 2:

- Implementar la interfaz GameModeStrategy en PlayerVsMachine.java.
- Crear el hook en checkLevelCompletion de PlayerVsMachine.java.
- Programar el "Perfil Aleatorio" (movimiento usando Math.random()).
- Diseñar y programar el "Perfil Experto" (Pathfinding/A*).

